

Your Agent Escaped Without Escaping the Sandbox

The agent changes the answers.
The check goes green.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–18 are for questions.

1. What changed when the test went green?

00:00-01:15

Show FAIL, then PASS. Ask what changed.

What changed when the test went green?

In an agent pipeline, a green check is what the next step trusts. So let's take one green check apart, byte by byte. I'm Yossi Eliaz, from Incredibuild.

We recorded one worker run in a Linux sandbox. The checker says FAIL before the worker's step, and PASS after it. For the rest of this talk we stay on that one pair of results, and we find the one thing that changed.

The setup is deliberately small. The worker is a script, not a model. The program is a few lines of command-line code that mimic an access check and print a status code. No web server is involved. Small on purpose: everything that matters fits on a slide, and every hash I show you is in the repository.

2. No login prints 200

01:15-02:00

Name the case. Then the two numbers.

One case is enough. admin:none means the admin route with no login. The right answer is 401. This program prints 200, for this case and for every other input.

A checker compares the program's output with an answer key of five cases. Only two of those five cases expect 200: a real admin, and the public route. This program says 200 five times, so it is wrong on the other three.

Keep this program in mind. Its bytes never change.

3. The checker is protected

02:00-03:00

Honest FAIL. DENIED. Same hash. Then PASS. Pause.

At the start, the answer key is honest. On this program, the checker prints FAIL and exits 1.

Then the worker takes its step. First it tries the obvious attack: overwrite the checker with one that just prints PASS. The checker sits on a read-only mount. DENIED. Its hash is the same before that step and after it.

Then the same checker runs again, launched the same way, on the same program. It prints PASS and exits 0.

Nothing we protected changed. Something else did.

4. Only the answer key moved

03:00-04:30

Read across the row. Stop on PASS.

Two rows: the honest run, then the edited run. Read across.

The program has the same hash both times. So does the checker. The answer key, `expected.json`, changes: 0494 when honest, 390e when edited. And the verdict goes from FAIL to PASS.

Underneath is the diff. For `admin:none`, the expected answer was 401. The worker rewrote it to 200. It did the same for the two other cases that expected an error. Now every expected answer is 200, which is exactly what the bad program prints.

The checker did its job perfectly. It compared the program with the answer key, and they matched. The worker did not change the checker. It changed what counted as correct.

5. What the three hashes say

04:30-05:30

One sentence per hash.

Each hash rules one thing out. Same program hash: the worker did not fix the code. Same checker hash: the checker's code did not change. New answer key hash: that is the input that moved.

Now the trap. The tempting fix is to hash the answer key. But hash it after the worker wrote it, and you have a perfect fingerprint of the worker's answers. A hash tells you which bytes were used. It does not tell you who had the right to write them. The answer key that decides has to belong to someone other than the worker, and live where the worker cannot write.

6. The file defines success

05:30-07:00

Point at the load. Then at who can write it.

So why did the checker trust that file? Because of this one line. The checker loads `expected.json` from its working directory. The file looks like ordinary project data. Nothing about it is special, except that the checker obeys it.

The worker can write this file. The checker treats it as the answer key. So the worker's write permission included the acceptance criteria. You can lock every executable in the pipeline and still leave this one

input on the worker's side of the line.

One honest limit. This recording shows the worker is allowed to rewrite the file. It does not measure whether a model would choose to. For context, Anthropic's November 2025 study of reward hacking first told models about specific hacks, then trained them on real coding tasks, and they learned to use them. One hack is in the same family as ours: a confest file that makes pytest report failures as passes. Its results are its own, not ours. This recording isolates the permission such a shortcut needs. One you already granted.

7. Controller-owned criteria

07:00-08:00

Say REJECT first. Then who owns the answers.

Now the repair. Same program, the one that prints 200 for everything. REJECT.

What changed is who owns the answers. The checker in the failing run also had a fresh sandbox of its own, with the workspace mounted read-only and no network, and it still went green, because it read its answers from the workspace. The controller runs the same program in a sandbox that holds only the program, reads back the output, and compares it in its own process against cases it holds itself. It never reads expected.json.

The worker's edit is still sitting in the workspace. It just no longer counts.

8. The corrected program

08:00-08:45

Different hash. Five numbers. ACCEPT.

A repair also has to let real work through. Here is the corrected program, with a different hash. It returns 401 for no login, 200 for a real admin, 403 for an ordinary user, 200 for the public route, and 401 for an expired login. Same controller-owned cases. ACCEPT.

That is the pair you want from any fix. The bad program is rejected, the good program is accepted, and the answers are owned by someone other than the worker.

9. One unused approval

08:45-10:15

Say unused. Then denied, denied, published, denied.

Accepting a program is not the same as shipping it. Between the verdict and the release, there is one more place to swap bytes.

The controller issues one approval for the corrected program. First, the bad program is offered under that fresh, unused approval. DENIED. Next, the approved bytes are pointed at a different destination. DENIED, because an approval names one place. Then the approved bytes go to the named place. PUBLISHED. Then the same bytes again. DENIED, because the approval is spent.

The first row is the one that matters. The approval was still unused, so the refusal came from checking the bytes, not from replay protection. An earlier version of our own test tried the bad bytes only after the approval was spent, so it could not tell which check refused them.

Publication here is a local store, a stand-in for a registry or a merge. It holds exactly the approved bytes.

10. Three questions for another system

10:15-11:45

Read the three questions. Do not name a vendor.

Take three questions home. Who can change the inputs that define acceptance? Which exact bytes were the candidate, the checker, and the criteria? And does publication store exactly the approved bytes?

The first question has a concrete starting point. List everything your acceptance step reads or looks for: files, config, environment, services. List what your agent can write. Any overlap other than the candidate itself is your finding. And keep the candidate out of the checker's process, so it can only answer, not grade. Then add a tripwire, not a policy: hash the answer key when the task starts and again at the verdict. On this recording, that hash moved.

None of that needs model telemetry. It will not tell you a change was malicious, only that the worker wrote what judged it. And these questions inspect a pipeline. On their own, they do not show that any particular product has this problem.

11. The sandbox held

11:45-13:15

Say what did not happen. Then name the allowed write.

Back to the title: your agent escaped without escaping the sandbox. The sandbox part is literal. Nothing got out. The checker overwrite was denied. The program bytes and the checker bytes never changed. The worker used exactly the write access it was supposed to have.

The allowed workspace write changed the decision. The answer key was in the workspace, and the checker obeyed it. That is the escape in the title: the worker never left the sandbox, it escaped the check.

So isolation and acceptance are different boundaries. A sandbox answers one question: what can this process touch? Acceptance asks another: can anything it touches decide that its own work is done? In a real repository, that answer key lives in the tests, the snapshots and the CI config, and writing tests is part of the agent's job. So taking the write away is not the fix. The agent may propose what counts as correct. It must never be the last writer of what judges it.

12. The worker changed what counted as correct

13:15-14:00

Repository first. Then the question, and its three-part answer. Thank you. Leave PASS up.

The code and the Linux recordings are in the repository on the screen. Every hash on these slides is in there, and you can rerun the whole thing on a disposable Linux machine with bubblewrap.

So, what changed when the test went green?

Not the program. Not the checker. The answer key.

Thank you.

Questions

13. The five policy cases

Questions

Use only in questions.

The handler takes a route and an authentication label and prints an integer. The runner checks these five. The repaired handler matches all five. This is the model of the check that promotes into the prod sandbox.

14. Runner token and the dev disk

Questions

Use only in questions.

The token case is loopback HTTP. The broad fixture stands in for the runner secret. The task fixture stands in for a job-scoped token. The mount case watches the host file after an ordinary write from the dev sandbox. This job's output stays writable.

15. Allowed host, wrong account

Questions

Use only in questions.

The fixture service permits both account paths, the way one allowed deploy host holds more than one operation. The delivery record shows the bytes arrived. The repair moves the POST onto the runner and removes the dev sandbox's direct route.

16. An earlier promote stored a marker

Questions

Use only in questions.

The archived case mutates a controller-side object, keeps a stale digest, and the old gate records a publication marker. The current promote derives the digest from immutable bytes and stores those bytes. Reaching the old flaw takes access to the controller object.

17. What was recorded

Questions

Use only in questions.

The records bind their source. The Linux recordings use bubblewrap on an Ubuntu machine. The worker under test is a script. Publication in the recording is the controller storing the exact approved bytes in a local database, a stand-in for a registry or a merge. Real-repository, multi-agent and snapshot effects are inferences from the answer-key dependency, not measurements. The timings are a delivery budget.

18. Reproduce the factory

Questions

Use only in questions.

Run the fixtures on disposable Linux with bubblewrap. The deck reads those recordings. Poisoned pipeline execution, OWASP CICD-SEC-4, is the public name for a CI job made to run commands an attacker changed in the repository. Ours is its data-only neighbor: the checker ran unchanged and obeyed a file the worker rewrote. Anthropic's reward-hacking study is prior motivation and its measurements stay with that study. Reproducing the talk needs no GitHub token.